

# KVM-Based Userspace Hypervisor: CPU Execution Modes, Trap-Driven Hypercalls, and Inter-VM Coordination

*A Systems Implementation of Hardware Virtualization, Memory Segmentation, Page Tables, and Multi-VM Communication*

**Pratik Wadekar**

Department of Computer Science & Engineering  
Systems Engineering Technical Report

## Abstract

Modern system virtualization relies on hardware extensions (such as Intel VMX and AMD-V) to execute guest virtual machines directly on physical processors while trapping privileged operations to a hypervisor. This report presents the design and implementation of a userspace Virtual Machine Monitor (VMM) developed directly on top of the Linux Kernel-based Virtual Machine (KVM) interface (`/dev/kvm`). The project is realized across two major architectural stages: (1) building a single-VM hypervisor capable of configuring, booting, and managing guests across four fundamental x86 execution modes (16-bit Real Mode, 32-bit Protected Mode, 32-bit Paged Mode, and 64-bit Long Mode) with an I/O port trap-driven hypercall interface; and (2) implementing a dual-VM environment orchestrating a producer-consumer communication model with hypervisor-mediated memory synchronization and deterministic scheduling. The report details the mechanics of KVM file-descriptor hierarchies, segment descriptors, multi-level page table construction, address translation bridging (GVA to HVA), and virtualization boundary handling.

## 1. INTRODUCTION AND SYSTEMS MOTIVATION

Hypervisors decouple operating systems from physical hardware, enabling multi-tenant cloud computing, workload consolidation, and secure isolation. While early virtualization employed complex software binary translation (trap-and-emulate), modern platforms rely on hardware virtualization extensions (Intel VMX / AMD-V) managed through kernel interfaces.

The Linux Kernel-based Virtual Machine (KVM) turns the Linux kernel into a Type-2 hypervisor by exposing hardware virtualization capabilities through the `/dev/kvm` character device. However, KVM only manages CPU virtualization and hardware page tables; all user-space device emulation, memory allocation, CPU register initialization, and guest lifecycle management must be provided by a userspace Virtual Machine Monitor (VMM).

The primary objective of this project is to construct a lightweight, userspace hypervisor from scratch to explore:

- **Low-Level KVM Interface:** Operating directly on `/dev/kvm` via system `ioctl` primitives without heavy management frameworks.
- **x86 Architectural Evolution:** Manually building register states, segment descriptors, and hierarchical page tables for Real, Protected, 32-bit Paged, and 64-bit Long modes.
- **Trap-Driven Hypercall Interfaces:** Exploiting I/O port instruction exits (`KVM_EXIT_IO`) to create synchronous guest-to-host control pathways.
- **Inter-VM Coordination:** Designing a hypervisor-mediated shared buffer and scheduler to orchestrate concurrent, isolated virtual machines.

## 2. VIRTUAL MACHINE INITIALIZATION AND MEMORY LAYOUT

The hypervisor interacts with KVM through a three-tiered file descriptor hierarchy: System FD  $\rightarrow$  VM FD  $\rightarrow$  vCPU FD.

### 2.1 KVM Initialization Protocol

The VMM initializes a guest instance through a deterministic sequence of system calls (`simple-kvm.c`):

1. **Open KVM Driver:** Opens `/dev/kvm` and verifies the API version using `ioctl(dev_fd, KVM_GET_API_VERSION, 0)` (expecting version 12).
2. **Create VM Instance:** Calls `ioctl(dev_fd, KVM_CREATE_VM, 0)` to allocate a kernel VM context.
3. **Configure Task State Segment (TSS):** Calls `ioctl(vm_fd, KVM_SET_TSS_ADDR, 0xffffbd000)` to designate a 3-page physical region required by Intel VMX for emulated real-mode operation.
4. **Guest Physical Memory Mapping:** Allocates 2 MB of host virtual memory via `mmap(NULL, 0x200000, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS | MAP_NORESERVE, -1, 0)`. The allocated memory is registered with the kernel via `KVM_SET_USER_MEMORY_REGION`, mapping guest physical address `0x0` to the host allocated pointer `vm->mem`.
5. **Create Virtual CPU:** Invokes `ioctl(vm_fd, KVM_CREATE_VCPU, 0)` to instantiate the virtual processor.
6. **Map Exit Communication Structure:** Queries `KVM_GET_VCPU_MMAP_SIZE` and maps the shared kernel-user `struct kvm_run` structure using `MAP_SHARED`.

### 2.2 Guest Physical Address Space Organization

The 2 MB guest physical memory region is structured with deterministic base offsets:

- **Guest Code Base (`0x0000`):** Entry point where compiled bare-metal guest binary payloads are loaded (`RIP = 0x0`).
- **Result Validation Address (`0x0400`):** Dedicated memory location where guest programs record test assertions (`mem[0x400] = 42`) prior to termination.

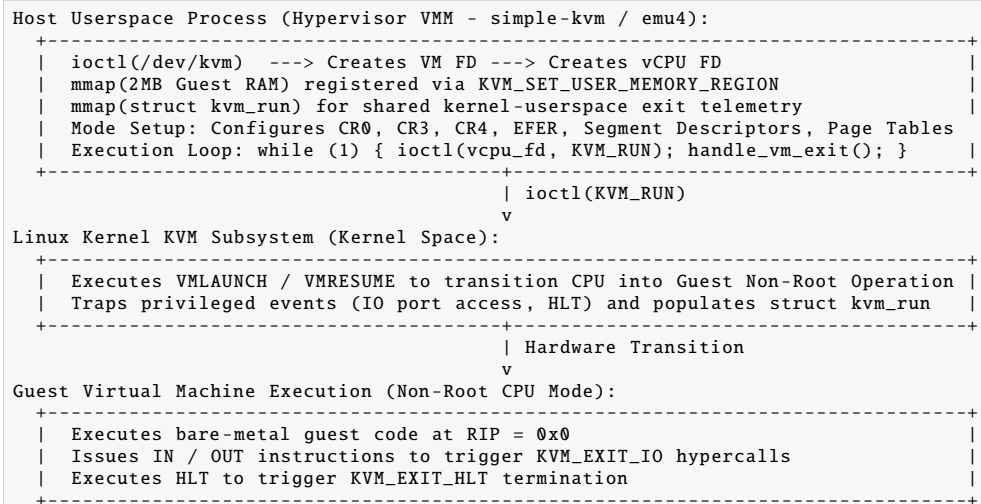


Figure 1: System Architecture: Host Userspace VMM, Linux KVM Kernel Module, and Hardware Guest vCPU

- **Page Table Hierarchy (0x2000 – 0x4000):** Physical root locations for Page Directory entries in 32-bit paged mode and PML4/PDPT/PD structures in 64-bit long mode.
- **I/O Buffer Area (0x7000):** Scratch region utilized for guest string transmission hypercalls.
- **Initial Stack Pointer (0x200000):** Stack base initialized to the top of the 2 MB allocation, growing downward.

### 3. x86 CPU EXECUTION MODE CONFIGURATION

The hypervisor manually constructs the CPU architectural state for four distinct execution modes using `KVM_GET_SREGS`, `KVM_SET_SREGS`, and `KVM_SET_REGS`.

### 3.1 16-Bit Real Mode

In Real Mode, the processor uses 20-bit segmented physical addressing (Address = Base  $\times$  16 + Offset). The hypervisor initializes segment selectors (CS, DS, ES, FS, GS, SS) with base `0x0`, limit `0xffff`, and selector `0x0`. Control registers CR0, CR3, and CR4 are zeroed, leaving protection and paging disabled.

### 3.2 32-Bit Protected Mode

Protected Mode enables 32-bit linear addressing and segment-level memory protection. The hypervisor manually constructs flat segment descriptors in `sregs`:

```
struct kvm_segment seg = {
    .base = 0, .limit = 0xffffffff, .present = 1, .dpl = 0,
    .db = 1, .s = 1, .l = 0, .g = 1, .avl = 0
};
seg.type = 11; seg.selector = 1 << 3; sregs.cs = seg; // Code Segment
seg.type = 3; seg.selector = 2 << 3; sregs.ds = seg; // Data Segment
sregs.es = sregs.fs = sregs.gs = sregs.ss = seg;
sregs.cr0 |= 1; // Set CR0.PE (Protected Mode Enable)
```

Setting `CR0.PE = 1` activates protected mode without enabling paging, providing direct 1-to-1 linear-to-physical address translation across a flat 4GB space.

### 3.3 32-Bit Paged Mode

In Paged 32-bit Mode, linear addresses are translated to physical addresses via hardware page directories. To map the 2 MB physical memory efficiently without multi-level page tables, the hypervisor leverages Page Size Extensions (PSE):

```
uint32_t *pde = (uint32_t *) (vm->mem + 0x2000);
pde[0] = 0x0 | (1 << 0) | (1 << 1) | (1 << 7); // Present | Writable | Page Size (4MB)
sregs.cr3 = 0x2000;
sregs.cr4 |= (1 << 4); // Set CR4.PSE (Page Size Extension)
sregs.cr0 |= (1 << 0) | (1 << 31); // Set CR0.PE and CR0.PG (Paging Enable)
```

The single 4MB Page Directory Entry (PDE) maps linear addresses **0x0–0x3fffff** directly to physical memory.

### 3.4 64-Bit Long Mode

Long Mode enables 64-bit execution and 4-level paging. The hypervisor establishes a 3-level page table hierarchy rooted at PML4:

1. **PML4 Table (0x2000):** First entry points to the Page Directory Pointer Table (PDPT) at 0x3000 with Present and Writable flags set.
2. **PDPT Table (0x3000):** First entry points to the Page Directory (PD) at 0x4000.
3. **Page Directory (0x4000):** First entry maps a 2 MB superpage with Present, Writable, and Page Size (1 << 7) flags set.
4. **Extended Registers:** Enables Physical Address Extension (CR4.PAE = 1), Long Mode Enable and Active flags (EFER.LME = 1, EFER.LMA = 1), and configures the Code Segment descriptor with the 64-bit long attribute (.l = 1, .db = 0).

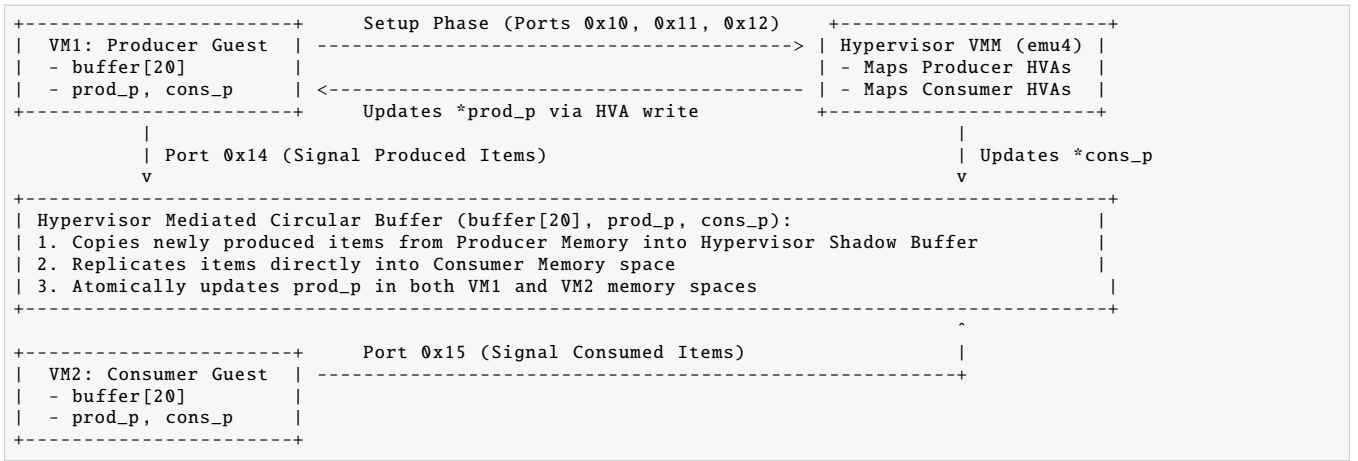


Figure 2: Mediated Multi-VM Architecture: Producer-Consumer Coordination and Shadow Buffer Replication

## 4. TRAP-DRIVEN HYPERCALL INTERFACE AND EXIT HANDLING

Because guest virtual machines operate in isolated CPU contexts without direct access to host system calls, communication requires generating trap-driven VM exits.

### 4.1 VM Exit Dispatching

The hypervisor enters the guest execution loop via `ioctl(vcpu->vcpu_fd, KVM_RUN, 0)`. The CPU transitions to guest non-root mode, executing until a privileged or intercepted instruction triggers a VM exit:

```

for (;;) {
    ioctl(vcpu->vcpu_fd, KVM_RUN, 0);
    switch (vcpu->kvm_run->exit_reason) {
        case KVM_EXIT_IO:
            handle_io_hypercall(vcpu);
            break;
        case KVM_EXIT_HLT:
            validate_and_terminate(vm, vcpu);
            return;
        default:
            fprintf(stderr, "Unhandled_exit_reason: %d\n", vcpu->kvm_run->exit_reason);
            exit(1);
    }
}

```

### 4.2 I/O Port Hypercall Protocol

The guest communicates by executing x86 IN and OUT assembly instructions against designated I/O ports. Data payloads are passed synchronously through the shared `struct kvm_run` memory region:

- **Port 0xE9 (Character Output):** The guest executes `outb(0xE9, char)`. The hypervisor reads data at offset `io.data_offset` from the shared run structure and prints to stdout.
- **Port 0xEA (Integer Output):** Prints a 32-bit unsigned integer formatted to stdout.
- **Port 0xEB (Integer Input):** The guest executes `in(0xEB)`. The hypervisor writes the integer into the shared I/O buffer before resuming.
- **Port 0xEC (Total Exit Query):** Returns the cumulative count of VM exits recorded by the hypervisor.
- **Port 0xED (String Output):** Reads a null-terminated string from guest physical memory starting at `0x7000`.
- **Port 0xEE/0xEF (Address Translation):** Translates Guest Virtual Addresses (GVA) to Host Virtual Addresses (HVA). The guest sends a GVA over port `0xEE`; the hypervisor queries `ioctl(vcpu_fd, KVM_TRANSLATE, &translation)`, computes `HVA = vm->mem + GPA`, and returns the pointer over port `0xEF`.

## 5. MULTI-VM COORDINATION: MEDIATED PRODUCER-CONSUMER MODEL

Part 2 of the project extends the hypervisor (`emu4.c`) to orchestrate two concurrent, isolated virtual machines executing a producer-consumer workload.

### 5.1 Address Space Registration Handshake

Because the two VMs reside in completely disjoint physical memory spaces (2 MB each), neither guest can address the other's memory directly. During the initialization handshake, both guests register their internal buffer coordinates with the hypervisor:

- **Port 0x10:** Transmits the guest buffer array GVA. The hypervisor computes the corresponding Host Virtual Address: `HVA = vm->mem + GVA`.
- **Port 0x11 / 0x12:** Transmits the GVAs of the producer index (`prod_p`) and consumer index (`cons_p`) pointers.

### 5.2 Hypervisor-Mediated Synchronization Protocol

The hypervisor maintains a shadow circular buffer of 20 elements (`uint32_t buffer[20]`). Data synchronization is driven strictly by I/O trap exits:

- **Producer Cycle (VM1 → Port 0x14):** The producer writes items to its local ring buffer and issues an OUT on port `0x14`

with its updated `prod.p`. The hypervisor reads newly added elements from VM1 memory, writes them to the hypervisor shadow buffer, copies them into VM2's consumer buffer, and updates the `prod.p` variables in both VM address spaces.

- **Consumer Cycle (VM2 → Port 0x15):** The consumer advances its local `cons.p` and issues an OUT on port 0x15. The hypervisor logs the consumed elements and synchronizes the updated `cons.p` value to both VM1 and VM2.

### 5.3 Deterministic Schedule Dispatcher

The execution order of the two VMs is dictated by an external schedule file (`sched1.txt`, `sched2.txt`) containing an array of VM identifiers ('1' or '2'). The hypervisor reads the schedule string and dispatches KVM\_RUN to the selected vCPU in round-robin or custom sequences, demonstrating deterministic virtual machine time-slicing.

## 6. SYSTEMS DEBUGGING AND IMPLEMENTATION CHALLENGES

### 6.1 Intel VMX Real-Mode Virtualization Restrictions

Initial hardware virtualization specifications in Intel VMX did not support 16-bit real mode in hardware, requiring hypervisors to execute real mode in software emulation. Modern processors introduce "Unrestricted Guest Mode," allowing arbitrary real and unpaged protected modes to execute natively under VMX. The hypervisor explicitly configures a dedicated Task State Segment (TSS) at `0xffffbd000` via `KVM_SET_TSS_ADDR` to satisfy VMX in-kernel emulation requirements.

### 6.2 Hypercall Trap Selection: VMCALL vs. HLT

While x86 provides dedicated `VMCALL` (Intel) and `VMMCALL` (AMD) instructions for guest-to-host hypercalls, KVM reserves these opcodes for internal kernel hypercall handling and does not surface them directly to userspace VMMs. To maintain a simple, reliable userspace interface, the hypervisor utilizes I/O port instructions (IN/OUT) for service requests and the HLT opcode for guest termination, both of which reliably trigger userspace VM exits (`KVM_EXIT_IO` and `KVM_EXIT_HLT`).

### 6.3 Address Translation Boundary Diagnostics

When testing KVM\_TRANSLATE functionality, querying unmapped or invalid virtual addresses (such as GVA `0xFFFFFFFF` in a 32-bit unpaged environment) causes the kernel translation ioctl to fail. The hypervisor handles translation failures gracefully by validating `translation.valid` flags and returning null pointer sentinels (`hva_global = 0`) to prevent host segmentation faults.

## 7. DESIGN TRADE-OFFS AND ARCHITECTURAL EXTENSIONS

- **Trap-Driven I/O vs. Shared Memory Channels:** Using I/O port instructions provides a straightforward hypercall mechanism without requiring complex shared-memory ring buffers. However, each operation incurs a full VM exit penalty (~ 1 000–10 000 CPU cycles). In higher-throughput systems, establishing shared host-guest memory regions with lockless ring buffers (e.g., VirtIO) significantly amortizes exit overhead.
- **Hypervisor Mediation vs. Direct Inter-VM Memory Sharing:** Mediating inter-VM communication through the hypervisor enforces strict physical isolation and simplifies access control. However, double-copying data (VM1 → Hypervisor → VM2) introduces latency compared to configuring shared physical pages via `KVM_SET_USER_MEMORY_REGION`.
- **Single-Threaded Dispatch vs. Multi-Threaded vCPUs:** The current implementation executes VMs sequentially in a single host thread. Transitioning to multi-threaded execution—assigning one POSIX thread per vCPU—would enable true parallel multi-core execution, requiring explicit mutex synchronization across shared hypervisor buffers.

## 8. REPRODUCTION AND EXECUTION GUIDE

The repository provides build scripts and binary targets for compiling bare-metal guest payloads and running hypervisor emulators:

- **Toolchain Requirements:** Linux kernel with `/dev/kvm` access, GCC with 32-bit multilib support (`gcc-multilib`), and GNU Binutils.
- **Part 1 Compilation and Execution:**

```
cd part1 && make
./simple-kvm -r # Execute 16-bit Real Mode Guest
./simple-kvm -s # Execute 32-bit Protected Mode Guest
./simple-kvm -p # Execute 32-bit Paged Mode Guest
./simple-kvm -l # Execute 64-bit Long Mode Guest
```

- **Part 2 Dual-VM Producer-Consumer Execution:**

```
cd part2 && make
./emu4 sched1.txt # Execute dual-VM scheduling with schedule sequence
```

## 9. CONCLUSION

The KVM-based userspace hypervisor illustrates the core principles of modern hardware-assisted virtualization. By interacting directly with the Linux KVM kernel API, the implementation demonstrates how a userspace VMM sets up guest physical address spaces, configures CPU registers across x86 execution modes, constructs hierarchical page tables, handles trap-driven hypercalls, and brokers communication across isolated virtual machines.